

مستند پروژه پایانی درس یادگیری تقویتی طراحی و تحلیل عامل هوشمند در هزارتوی پویا



مقدمه

دانشجویان گرامی، این پروژه با هدف پیوند دادن مباحث نظری یادگیری تقویتی با طراحی یک سامانه قابل اجرا تدوین شده است. شما باید یک محیط هزارتوی پویا طراحی کنید، آن را به صورت یک فرایند تصمیم‌گیری مارکوف مدل کنید و عملکرد سه الگوریتم متفاوت را روی شرایط یکسان بسنجید. بخش انتقال یادگیری، رابط گرافیکی و تحلیل آماری نتایج نیز باید به عنوان اجزای مکمل پروژه پیاده‌سازی شوند.

خروجی نهایی باید نشان دهد که شما علاوه بر اجرای کد، منطق تصمیم‌گیری عامل، تفاوت روش‌های مدل‌محور و مدل‌آزاد، اثر طراحی پاداش، نقش اکتشاف و بهره‌برداری، مفهوم ردپای شایستگی و حدود استفاده از انتقال دانش را درک کرده‌اید. صرف اجرای یک نمونه آماده یا نمایش چند نمودار بدون تحلیل، پاسخ پروژه محسوب نمی‌شود.

چارچوب کلی پروژه

پروژه به صورت انفرادی انجام می‌شود و مهلت آن تا تاریخ مقرر شده است. محیط، داده‌های خام، الگوریتم‌ها، نمودارها، رابط گرافیکی، مستندات و گزارش نهایی باید در یک مخزن منظم نگهداری شوند. همه نتایج باید قابل بازتولید باشند؛ یعنی با اجرای دستورهای درج‌شده در فایل راهنما، همان نقشه‌ها و آزمایش‌ها دوباره تولید شوند.

خلاصه پروژه

یک هزارتوی پویا با نقشه اختصاصی بسازید، سه الگوریتم Q-Learning، Value Iteration و SARSA(λ) را پیاده‌سازی و مقایسه کنید، یک بخش محدود انتقال یادگیری را روی Q-Learning اجرا کنید و نتایج را در قالب رابط گرافیکی، نقشه‌های حرارتی، سیاست‌های بصری، نمودارهای آماری و گزارش تحلیلی ارائه دهید.

الزامات GitHub

استفاده از **GitHub** اجباری است. نام مخزن باید به صورت **RL_FinalProject_<StudentID>** انتخاب شود و ساختار پوشه‌ها مطابق بخش «ساختار تحویل» تنظیم گردد. حداقل سه **commit** معتبر ثبت شود؛ به گونه‌ای که روند توسعه محیط، الگوریتم‌ها، آزمایش‌ها و رابط گرافیکی قابل مشاهده باشد و repository باید **public** باشد.

تعریف مسئله و محیط هزارتو

محیط اصلی یک هزارتوی دوبعدی با اندازه‌ای بین 15×15 تا 18×18 خانه است. عامل از یک نقطه مشخص آغاز می‌کند، ابتدا باید کلید را به دست آورد، سپس به در خروجی برسد و در نهایت وارد خانه هدف شود. محیط باید شامل دیوار، خانه عادی، خانه جریمه، نقطه شروع، کلید، در بسته و هدف باشد. حداقل ۱۵ درصد خانه‌ها باید مانع و حداقل پنج خانه باید جریمه‌دار باشند.

در هر گام، عامل یکی از چهار عمل بالا، پایین، چپ یا راست را انتخاب می‌کند. با احتمال ۰,۸ عمل انتخاب‌شده اجرا می‌شود و با احتمال ۰,۱ عامل به هر یک از دو جهت عمود منحرّف می‌گردد. برخورد با دیوار باعث باقی ماندن عامل در حالت فعلی و دریافت جریمه می‌شود. این عدم قطعیت باید در مدل انتقال و آزمایش‌های هر سه الگوریتم لحاظ شود.

تعریف حالت و حفظ خاصیت مارکوف

در تعریف پایه، حالت فقط مختصات مکانی نیست؛ زیرا باز یا بسته بودن در به دریافت کلید وابسته است. بنابراین حداقل نمایش حالت به‌صورت زیر در نظر گرفته می‌شود. متغیر k برابر صفر یا یک است و وضعیت دریافت کلید را مشخص می‌کند.

$$s = (x, y, k)$$

در طراحی محیط هزارتو، علاوه بر عناصر اصلی شامل دیوار، خانه‌های عادی، نقطه شروع، کلید، در خروجی و هدف، باید حداقل یکی از قابلیت‌های زیر نیز پیاده‌سازی شود:

انرژی محدود، خانه لغزنده، انتقال‌دهنده، مانع متحرک، دروازه دوره‌ای یا هدف متغیر.

قابلیت انتخاب‌شده باید بر رفتار عامل و فرایند تصمیم‌گیری آن تأثیر واقعی داشته باشد و صرفاً جنبه نمایشی نداشته باشد. همچنین لازم است این قابلیت در رابط گرافیکی محیط به‌صورت مشخص نمایش داده شود و نحوه عملکرد آن در گزارش پروژه توضیح داده شود.

تعریف حالت محیط باید متناسب با قابلیت انتخاب‌شده انجام گیرد. برای مثال، در صورت انتخاب انرژی محدود، مقدار انرژی باقی‌مانده باید بخشی از حالت باشد. در صورت استفاده از مانع متحرک یا هدف متغیر نیز موقعیت فعلی مانع یا هدف باید در حالت عامل در نظر گرفته شود. حالت باید به‌گونه‌ای تعریف شود که با دانستن حالت فعلی و عمل انتخاب‌شده، بتوان رفتار مرحله بعد محیط را مشخص کرد و نیازی به بررسی تاریخچه مراحل قبلی نباشد.

Seed اختصاصی و تولید نقشه

رقم یکی‌مانده به‌آخر شماره دانشجویی، Seed پایه پروژه است. برای نمونه، در شماره دانشجویی 40123456 مقدار Seed برابر ۵ خواهد بود. اندازه نقشه نیز با استفاده از همین مقدار تعیین می‌شود.

$$b = \text{int}(\text{StudentID}[-2]), \quad N = 15 + (b \bmod 4)$$

```
student_id = '40123456'  
base_seed = int(student_id[-2])  
maze_size = 15 + (base_seed %4)
```

پس از تولید نقشه، باید با یک الگوریتم جست‌وجوی قطعی مانند BFS بررسی شود که مسیر معتبری از شروع به کلید و سپس از کلید به هدف وجود دارد. اگر نقشه نامعتبر بود، نقشه با روشی مشخص و تکرارپذیر اصلاح یا دوباره تولید شود. فایل نهایی نقشه باید ذخیره شود تا تمام الگوریتم‌ها دقیقاً روی یک محیط یکسان اجرا شوند.

مدل‌سازی MDP و تابع پاداش

پیش از پیاده‌سازی الگوریتم‌ها، مسئله باید به‌صورت یک فرایند تصمیم‌گیری مارکوف تعریف شود. گزارش باید فضای حالت، فضای عمل، تابع انتقال، تابع پاداش، ضریب کاهش، حالت‌های پایانی و سیاست عامل را به‌صورت دقیق توضیح دهد. تعریف MDP باید با کد محیط سازگار باشد و هیچ متغیر مؤثر بر آینده از نمایش حالت حذف نشده باشد.

طراحی دو نسخه از تابع پاداش

نسخه نخست، پاداش spars است. در این نسخه بخش عمده پاداش فقط هنگام دریافت کلید یا رسیدن به هدف داده می‌شود و هر حرکت هزینه کوچکی دارد. نسخه دوم باید از Reward Shaping استفاده کند و برای رفتارهای میانی مانند نزدیک شدن منطقی به کلید، عبور ایمن از ناحیه خطرناک یا کاهش اتلاف حرکت بازخورد مناسب در نظر بگیرد.

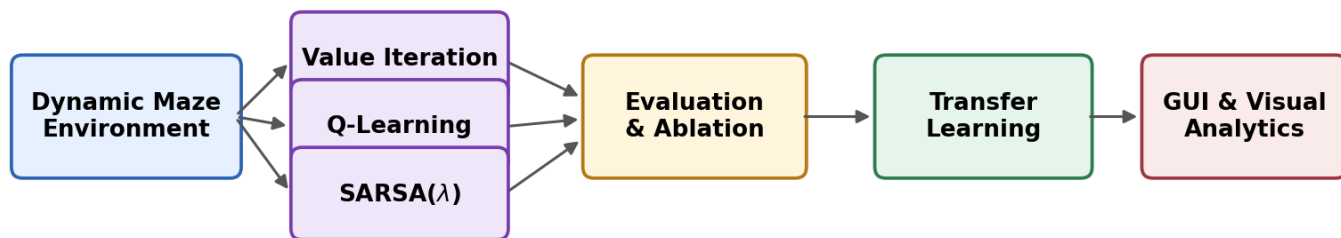
مقادیر دقیق پاداش را شما انتخاب می‌کنید، اما باید برای انتخاب خود استدلال ارائه دهید. همچنین لازم است نشان داده شود که آیا پاداش شکل‌دهی شده صرفاً آموزش را سریع‌تر کرده است یا سیاست نهایی را نیز تغییر داده و آیا رفتار ناخواسته‌ای مانند حرکت حلقه‌ای، جمع‌آوری پاداش فرعی بدون تکمیل مأموریت یا اجتناب بیش از حد از خانه‌های پرخطر ایجاد شده است.

حداقل رویدادهای قابل ثبت

حرکت عادی، برخورد با دیوار، ورود به خانه جریمه، دریافت کلید، تلاش برای عبور از در بسته، عبور موفق از در، رسیدن به هدف و پایان اپیزود به علت رسیدن به سقف تعداد گام‌ها باید در فایل لاگ ذخیره شوند.

سقف اپیزود و شرط پایان

هر اپیزود با رسیدن عامل به هدف، تمام شدن انرژی در صورت انتخاب قابلیت انرژی محدود، یا عبور تعداد گام‌ها از سقف تعیین‌شده پایان می‌یابد. سقف پیشنهادی برابر سه برابر تعداد خانه‌های قابل عبور است. انتخاب مقدار نهایی باید در فایل تنظیمات پروژه ثبت شود.



نمای کلی اجزای فنی پروژه

الگوریتم اول: Value Iteration

در این بخش، مدل کامل محیط شامل احتمال انتقال و تابع پاداش در اختیار الگوریتم قرار می‌گیرد. Value Iteration باید بدون استفاده از پیاده‌سازی آماده کتابخانه‌ها نوشته شود و برای محاسبه تابع ارزش بهینه و استخراج سیاست بهینه به کار رود.

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

شرط همگرایی باید بر اساس حداکثر تغییر تابع ارزش در دو تکرار متوالی تعریف شود. مقدار آستانه، ضریب کاهش، تعداد تکرارها، زمان اجرا و سیاست نهایی ذخیره شوند. سیاست حاصل از این الگوریتم مرجع مقایسه برای دو روش مدل‌آزاد خواهد بود.

- پیاده‌سازی مستقل به‌روزرسانی بلمن و استخراج سیاست حریصانه
- نمایش نقشه حرارتی تابع ارزش و فلش بهترین عمل در هر حالت
- گزارش تعداد تکرار تا همگرایی و زمان اجرا
- تحلیل اثر حداقل سه مقدار متفاوت ضریب تنزیل γ

الگوریتم دوم: Q-Learning

Q-Learning به‌عنوان روش off-policy اجرا می‌شود. سیاست رفتار باید ϵ -greedy باشد و مقدار ϵ در طول آموزش کاهش پیدا کند. باید دست‌کم دو برنامه کاهش ϵ ، مانند کاهش خطی و نمایی، را پیاده‌سازی و مقایسه کنید.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

برای تحلیل درست، مقادیر γ ، α و ϵ در فایل تنظیمات ثبت می‌شوند. علاوه بر منحنی پاداش، تعداد گام، نرخ موفقیت، برخورد با دیوار و تعداد ورود به خانه جریمه باید در هر اپیزود ذخیره شود. حداقل یک به‌روزرسانی واقعی Q باید از فایل لاگ انتخاب و به‌صورت دستی در گزارش بازسازی گردد.

الگوریتم سوم: SARSA(λ)

در الگوریتم سوم، یادگیری on-policy همراه با Eligibility Trace پیاده‌سازی می‌شود. نوع ان می‌تواند accumulating یا replacing باشد، اما علت انتخاب آن باید در گزارش توضیح داده شود. مقدار λ برای ۰،۳، ۰،۷ و ۰،۹ آزمایش گردد.

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t), \quad Q \leftarrow Q + \alpha \delta_t E$$

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + \mathbf{1}\{s = s_t, a = a_t\}$$

باید توضیح داده شود که $\lambda=0$ چگونه روش را به SARSA یک مرحله‌ای نزدیک می‌کند و افزایش λ چگونه خطای TD را به حالت‌ها و اعمال قبلی منتقل می‌سازد. برای دست‌کم یک اپیزود کوتاه، تغییرات δ و E در چند گام متوالی ثبت و تفسیر شود.

مقایسه سه الگوریتم

هر سه الگوریتم باید روی یک نقشه و یک تعریف پاداش یکسان مقایسه شوند. مقایسه فقط به پاداش نهایی محدود نیست و باید زمان اجرا، تعداد نمونه موردنیاز، پایداری بین اجراها، حافظه مصرفی، کیفیت مسیر و حساسیت به پارامترها را نیز در بر گیرد.

برای سنجش کیفیت سیاست، درصد حالت‌هایی محاسبه شود که عمل حریصانه Q-Learning یا SARSA(λ) با عمل بهینه Value Iteration یکسان است. اختلاف‌ها باید در یک نقشه رنگی نمایش داده شوند و دست‌کم سه حالت نمونه با توجه به ساختار محلی محیط تحلیل شوند.

SARSA(λ)	Q-Learning	Value Iteration	معیار
Model-free, on-policy	Model-free, off-policy	model -base	نوع روش
اپیزود	اپیزود	تکرار بلمن	واحد پیشرفت
Q، E و سیاست	Q و سیاست	V و سیاست	خروجی اصلی
اثر λ و رفتار ایمن	نمونه، پاداش و پایداری	زمان و همگرایی	مقایسه اجباری

بخش انتقال یادگیری

این بخش فقط بر روی Q-Learning انجام می‌گیرد. ابتدا عامل در محیط مبدأ آموزش می‌بیند و جدول Q آن ذخیره می‌شود. سپس دو محیط مقصد، یکی با تغییرات محدود و دیگری با تغییرات گسترده، ساخته خواهند شد.

محیط‌های مقصد

در محیط مقصد مشابه، حدود ۱۵ تا ۲۰ درصد موانع جابه‌جا می‌شوند، اما نقطه شروع، کلید و هدف ثابت می‌مانند. در محیط مقصد متفاوت، دست‌کم ۳۵ درصد موانع تغییر می‌کنند، محل کلید یا هدف جابه‌جا می‌شود و چند خانه جریمه جدید افزوده می‌گردد. هر دو نقشه باید با BFS از نظر وجود مسیر معتبر کنترل شوند.

چهار سناریوی آموزش

- 1) آموزش از صفر با جدول Q اولیه صفر؛ این حالت خط مبنا است.
- 2) انتقال کامل تمام جدول Q محیط مبدأ به محیط مقصد.
- 3) انتقال تعدیل‌شده با ضریب β برای کنترل شدت انتقال.
- 4) انتقال انتخابی؛ فقط حالت‌هایی منتقل شوند که همسایگی محلی آن‌ها در دو محیط تغییر نکرده است.

$$Q_T^{(0)}(s, a) = \beta Q_S(s, a), \quad \beta \in \{0.25, 0.50, 0.75\}$$

برای هر محیط مقصد باید عملکرد اولیه، سرعت یادگیری و عملکرد نهایی جداگانه گزارش شود. همچنین حداقل یک نمونه انتقال منفی پیدا شود که در آن دانش منتقل‌شده عامل را به عمل نامناسب هدایت کرده است. مقادیر Q آن حالت، تغییر ساختار محیط و نحوه اصلاح رفتار در ادامه آموزش باید نمایش داده شوند.

انتظار تحلیلی

گزارش باید میان انتقال مثبت، انتقال منفی، عملکرد آغازین، سرعت یادگیری و عملکرد نهایی تمایز قائل شود. جمله‌هایی مانند «انتقال خوب بود» بدون تعریف معیار و شواهد عددی قابل قبول نیست.

رابط گرافیکی و نمایش بصری

پروژه باید یک رابط گرافیکی مستقل و قابل استفاده داشته باشد. نمایش یک تصویر ثابت از نقشه کافی نیست. رابط می‌تواند با PyQt، Tkinter، Pygame یا ابزار مشابه ساخته شود و باید حرکت عامل را به صورت مرحله به مرحله نمایش دهد.

دیوارها، خانه‌های عادی، جریمه‌ها، کلید، در، هدف، عامل و قابلیت تکمیلی انتخاب شده باید نشانه‌های بصری متمایز داشته باشند. تغییر وضعیت کلید، باز شدن در، برخورد با مانع، ورود به خانه خطرناک و پایان موفق یا ناموفق اپیزود نیز باید در لحظه قابل مشاهده باشد.

کنترل‌های رابط

- انتخاب الگوریتم و محیط مبدأ یا مقصد
- انتخاب حالت آموزش یا ارزیابی
- شروع، توقف، ادامه، بازنشانی و اجرای مجدد
- کنترل سرعت انیمیشن
- فعال یا غیرفعال کردن نمایش سیاست
- نمایش اطلاعات لحظه‌ای شامل شماره اپیزود، تعداد گام، پاداش، ϵ ، وضعیت کلید و نرخ موفقیت اخیر

خروجی‌های بصری

پس از پایان آموزش، رابط یا بخش تحلیل باید بتواند Heatmap تابع ارزش، فلش سیاست نهایی، نقشه تعداد بازدید از حالت‌ها، مسیر نهایی عامل، نقاط اختلاف سیاست‌ها و تفاوت Q قبل و بعد از انتقال را نمایش دهد. خروجی‌ها باید در قالب عکس ذخیره شوند.

نمایش	حداقل محتوای مورد انتظار
Heatmap ارزش	مقدار V یا Q max برای تمام حالت‌های معتبر
سیاست نهایی	فلش بهترین عمل و نشانه حالت‌های پایانی
نقشه بازدید	تعداد مراجعه عامل به هر حالت در آموزش
اختلاف سیاست	حالت‌های موافق و مخالف با سیاست مرجع
انتقال یادگیری	تفاوت Q یا سیاست پیش و پس از انتقال

قاعده گزارش نمودارها

زیر هر نمودار حداقل یک پاراگراف تحلیلی نوشته شود. تحلیل باید علت روند، نوسان، شکست، تفاوت روش‌ها و محدودیت نتیجه را توضیح دهد؛ تکرار اعداد محورهای نمودار به‌تنهایی تحلیل محسوب نمی‌شود.

پرسش‌های تحلیلی گزارش

گزارش نهایی علاوه بر شرح پیاده‌سازی باید به پرسش‌های زیر پاسخ دهد. پاسخ‌ها باید به کد، فایل لاگ و نتایج واقعی پروژه ارجاع دهند و صرفاً تعریف کتابی نباشند.

- (1) مسئله را به‌صورت MDP کامل تعریف کنید و توضیح دهید نمایش حالت شما چگونه خاصیت مارکوف را حفظ می‌کند.
- (2) تفاوت on-policy و off-policy را با رفتار واقعی SARSA و Q-Learning در نزدیکی خانه‌های خطرناک توضیح دهید.
- (3) چرا Value Iteration به مدل انتقال نیاز دارد، اما دو الگوریتم دیگر بدون مدل محیط یاد می‌گیرند؟ مزیت و محدودیت هر رویکرد در پروژه شما چیست؟
- (4) کدام مقدار λ بهترین تعادل را میان سرعت یادگیری و پایداری ایجاد کرد؟ نتیجه را با شواهد عددی توضیح دهید.
- (5) سه حالت را پیدا کنید که سیاست مدل آزاد با سیاست Value Iteration متفاوت است و علت احتمالی اختلاف را تحلیل کنید.
- (6) در بخش انتقال، تفاوت محیط مشابه و متفاوت را از نظر عملکرد اولیه، سرعت یادگیری، عملکرد نهایی و انتقال منفی مقایسه کنید.

مخزن پروژه باید منظم، قابل اجرا و فاقد فایل‌های غیرضروری باشد. ساختار زیر پیشنهاد می‌شود و تغییر آن در صورت حفظ منطق جداسازی بخش‌ها مجاز است.

```
RL_FinalProject_<StudentID>/
├── environments/
│   ├── maze.py
│   ├── generator.py
│   └── maps/
├── agents/
│   ├── value_iteration.py
│   ├── q_learning.py
│   └── sarsa_lambda.py
├── transfer/transfer_learning.py
├── gui/
│   ├── app.py
│   └── renderer.py
├── experiments/
│   ├── run_experiments.py
│   ├── analysis.py
│   └── configs/
├── results/
│   ├── raw_data/
│   ├── models/
│   ├── figures/
│   └── videos/
├── tests/
├── report.pdf
├── requirements.txt
├── README.md
└── main.py
```

اجرای اصلی پروژه و اجرای آزمایش‌های حجیم باید از طریق دستورهای مشخص در README امکان‌پذیر باشند. مسیر فایل‌ها نباید به رایانه شخصی وابسته باشد.

```
python main.py
python experiments/run_experiments.py
```

موارد تحویلی

لینک مخزن GitHub شامل:

- گزارش نهایی PDF
- کد کامل محیط، سه الگوریتم و انتقال یادگیری
- داده‌های خام CSV و فایل تنظیمات هر آزمایش
- مدل‌ها یا جدول‌های Q ذخیره‌شده
- محتوای بصری
- README شامل روش نصب، اجرا و بازتولید نتایج
- فایل requirements.txt و حداقل چند تست واحد برای محیط و به‌روزرسانی‌ها

* تاریخچه commit ها برای نمره دهی بررسی خواهد شد.

استفاده از منابع و ابزارهای کمکی

استفاده از منابع آموزشی برای یادگیری مفاهیم و رفع خطا مجاز است، اما کد محیط، الگوریتم‌ها و تحلیل نتایج باید توسط دانشجو درک و توسعه داده شوند. هر بخش اقتباس‌شده باید در README یا گزارش با ذکر منبع مشخص شود. شباهت ساختاری یا متنی غیرعادی میان پروژه‌ها، به‌ویژه در نقشه، فایل‌های لاگ، تحلیل و خطاهای یکسان، بررسی خواهد شد.

استفاده از ابزارهای هوش مصنوعی به‌عنوان دستیار اشکال‌زدایی یا پیشنهاد اولیه ممنوع نیست، اما تحویل خروجی آماده بدون فهم و آزمایش قابل قبول نخواهد بود. در انتهای گزارش، جدولی کوتاه درج شود که موارد استفاده، پیشنهاد دریافت‌شده، تغییر انجام‌شده توسط دانشجو و دلیل اصلاح را نشان دهد. دست‌کم دو نمونه از پیشنهادهای ناقص یا ناسازگار باید توضیح داده شوند.

تمام نمودارها باید از داده‌های خام موجود در مخزن تولید شوند. دست‌کاری دستی نمودار، وارد کردن عدد بدون فایل اجرا، حذف اجرای ناموفق یا گزارش تنها بهترین Seed مجاز نیست. فایل تنظیمات هر آزمایش، زمان اجرا و نسخه کد مرتبط باید قابل ردیابی باشد.

نکات پایانی

- استفاده از پیاده‌سازی آماده الگوریتم‌های RL مانند Stable-Baselines یا RLlib مجاز نیست.
- استفاده از NumPy، Pandas، Matplotlib و کتابخانه رابط گرافیکی مجاز است.
- BFS فقط برای اعتبارسنجی نقشه به کار می‌رود و جایگزین عامل یادگیرنده نیست.
- گزارش باید تایپ‌شده، منظم و شامل شرح کد، نتایج، تحلیل و محدودیت‌ها باشد.
- هر پرسش یا ابهام باید پیش از هفته پایانی از طریق گروه ارتباطی درس مطرح شود.